What is EDA

EDA কী?

EDA এর পূর্ণরূপ হলো **Exploratory Data Analysis**।

বাংলায় একে বলা যায় ডেটা অনুসন্ধানমূলক বিশ্লেষণ।

Machine Learning বা Data Science প্রজেক্টে Model তৈরির আগে Dataset সম্পর্কে ভালোভাবে বোঝার জন্য যে বিশ্লেষণ করা হয়, সেটাই EDA।

সহজভাবে বললে, EDA হলো Dataset এর ভিতরে কী আছে, কোন ধরনের তথ্য আছে, কোথায় সমস্যা আছে, কোন Pattern আছে — এসব খুঁজে বের করার প্রক্রিয়া।

কেন EDA প্রয়োজন?

অনেক সময় Dataset দেখতে ঠিক মনে হলেও ভিতরে বিভিন্ন সমস্যা থাকতে পারে। যেমন:

- Missing Value থাকতে পারে
- ভুল Data Type থাকতে পারে
- Duplicate Data থাকতে পারে
- Outlier থাকতে পারে
- Irrelevant Feature থাকতে পারে
- Data Imbalance থাকতে পারে

যদি এসব সমস্যা আগে থেকে শনাক্ত না করা হয়, তাহলে Machine Learning Model ভালো Result দিতে পারবে না।

তাই EDA হলো Data Science Workflow এর সবচেয়ে গুরুত্বপূর্ণ ধাপগুলোর একটি।

EDA এর মূল উদ্দেশ্য

EDA করার সময় সাধারণত নিচের বিষয়গুলো বোঝার চেষ্টা করা হয়:

১. Dataset Structure বোঝা

- কতগুলো Row আছে
- কতগুলো Column আছে
- কোন Column Numerical
- কোন Column Categorical

২. Data Quality যাচাই করা

- Missing Value আছে কি না
- Duplicate আছে কি না
- Invalid Data আছে কি না

৩. Statistical Understanding তৈরি করা

- Mean
- Median
- Mode
- Standard Deviation
- Minimum ও Maximum Value

৪. Pattern এবং Relationship খুঁজে বের করা

- কোন Feature গুরুত্বপূর্ণ
- কোন Feature Target এর সাথে সম্পর্কিত
- কোন Feature অপ্রয়োজনীয়

৫. Visualization এর মাধ্যমে Data বোঝা

Graph ব্যবহার করে Data আরও সহজে বিশ্লেষণ করা যায়।

যেমন:

- Histogram
- Boxplot
- Countplot
- Scatterplot

Real Life Example

ধরুন আপনি একটি Student Dataset নিয়ে কাজ করছেন।

Dataset এ নিচের তথ্যগুলো আছে:

Name	Age	Gender	CGPA
Rahim	22	Male	3.75
Karima	21	Female	3.90

এখন Model তৈরির আগে আপনি জানতে চাইবেন:

- Average CGPA কত?
- Male/Female ratio কত?
- কোনো Missing Value আছে কি?
- Age এবং CGPA এর মধ্যে সম্পর্ক আছে কি?

এই পুরো Investigation Process-ই হলো EDA।

EDA এর প্রধান ধাপসমূহ

সাধারণত EDA কয়েকটি ধাপে করা হয়:

- Data Understanding
- Data Cleaning
- Feature Screening
- Statistical Profiling
- Univariate Analysis
- Bivariate Analysis
- Multivariate Analysis
- Data Visualization

EDA এর ধরন

EDA সাধারণত দুই ধরনের Analysis নিয়ে কাজ করে:

১. Univariate Analysis

একটি Column আলাদাভাবে বিশ্লেষণ করা।

উদাহরণ:

- Age distribution
- Gender count

২. Multivariate Analysis

একাধিক Column এর মধ্যে সম্পর্ক বিশ্লেষণ করা।

উদাহরণ:

- Age vs Salary
- Study Hour vs CGPA

EDA এবং Machine Learning

EDA ছাড়া ভালো Machine Learning Model তৈরি করা কঠিন।

কারণ EDA:

- Data সম্পর্কে ধারণা দেয়
- Feature Selection সহজ করে
- Data Cleaning করতে সাহায্য করে
- Better Model Performance নিশ্চিত করে

গুরুত্বপূর্ণ বিষয়

EDA শুধুমাত্র Graph আঁকার কাজ নয়।

অনেকেই মনে করে EDA মানেই Visualization।

কিন্তু বাস্তবে EDA হলো:

- Data বোঝা
- Data যাচাই করা
- Statistical Analysis করা
- Pattern খুঁজে বের করা
- Business Insight বের করা

Visualization হচ্ছে EDA এর একটি অংশ মাত্র।

সারাংশ

EDA হলো Dataset কে গভীরভাবে বোঝার প্রক্রিয়া।

এর মাধ্যমে আমরা:

- Data Structure বুঝি
- Problem খুঁজে বের করি
- Important Feature শনাক্ত করি
- Pattern আবিষ্কার করি
- Model তৈরির জন্য Data প্রস্তুত করি

তাই বলা যায়:

“EDA is the process of understanding data before building any Machine Learning model.”

Analytical View of a Dataset

Analytical View of a Dataset কী?

EDA শুরু করার পর প্রথম কাজগুলোর একটি হলো Dataset-কে বিশ্লেষণধর্মীভাবে দেখা বা বুঝা।
এটিকেই বলা হয় **Analytical View of a Dataset**।
এই ধাপে আমরা Dataset সম্পর্কে প্রাথমিক ধারণা নেওয়ার চেষ্টা করি।

যেমন:

- Dataset এ কতগুলো Row আছে
- কতগুলো Column আছে
- কোন ধরনের Feature আছে
- Data কেমন দেখাচ্ছে
- Sample Data কেমন
- Dataset বড় নাকি ছোট

এই ধাপটি অত্যন্ত গুরুত্বপূর্ণ কারণ Dataset সম্পর্কে পরিষ্কার ধারণা ছাড়া পরবর্তী Analysis সঠিকভাবে করা কঠিন।

প্রয়োজনীয় লাইব্রেরি Import

প্রথমে প্রয়োজনীয় Python Library Import করা হয়।

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Dataset Load করা

এখন CSV File থেকে Dataset Load করা হবে।

```
df = pd.read_csv('titanic_data_updated.csv')
```

এখানে:

- pd.read_csv() ব্যবহার করে CSV File পড়া হয়েছে
- Dataset টি df নামে একটি DataFrame এ রাখা হয়েছে

Dataset এর Shape দেখা

Dataset এ কতগুলো Row এবং Column আছে তা দেখার জন্য shape ব্যবহার করা হয়।

```
df.shape
```

ধরুন Output এসেছে:

```
(891, 12)
```

এখানে:

- 891 = Total Rows
- 12 = Total Columns

অর্থাৎ Dataset এ:

891 টি Sample Data আছে
12 টি Feature আছে

Total Feature এবং Sample Data বের করা

এখন আলাদাভাবে Total Feature এবং Total Sample Data দেখা যাক।

```
print(f"Total potential features :{df.shape[1]}")
```

```
print(f"Total sample data :{df.shape[0]}")
```

সম্ভাব্য Output:

```
Total potential features :12
```

```
Total sample data :891
```

এখানে:

df.shape[1] Column সংখ্যা দেয়
df.shape[0] Row সংখ্যা দেয়

কেন Shape গুরুত্বপূর্ণ?

Dataset এর Shape জানলে আমরা বুঝতে পারি:

Dataset বড় নাকি ছোট
কত Feature নিয়ে কাজ করতে হবে
Computational Complexity কেমন হতে পারে
Data যথেষ্ট আছে কি না

উদাহরণ:
যদি Dataset এ:

খুব কম Row থাকে → Model Underfit হতে পারে
খুব বেশি Feature থাকে → Feature Selection প্রয়োজন হতে পারে

Dataset এর প্রথম কিছু Row দেখা

Dataset এর শুরু থেকে কয়েকটি Row দেখার জন্য head() ব্যবহার করা হয়।

```
1df.head(10)
```

এখানে:

- প্রথম 10 টি Row দেখানো হবে

এই Method ব্যবহার করে আমরা বুঝতে পারি:

- Column Name কী
- Data Type কেমন হতে পারে
- Missing Value আছে কি না
- Numerical নাকি Categorical Data

কেন head() ব্যবহার করা হয়?

কারণ বড় Dataset পুরোটা একসাথে দেখা সম্ভব নয়।
তাই শুরুতে কয়েকটি Row দেখে Dataset সম্পর্কে Initial Understanding তৈরি করা হয়।

Random Sample দেখা

শুধু প্রথম Row দেখলে অনেক সময় Dataset সম্পর্কে ভুল ধারণা হতে পারে।
তাই Random Sample দেখাও গুরুত্বপূর্ণ।

```
1df.sample(10)
```

এখানে:

- Dataset থেকে Randomly 10 টি Row দেখানো হবে

sample() কেন গুরুত্বপূর্ণ?

কারণ অনেক সময় Dataset এর শুরু এবং শেষ অংশে Pattern আলাদা হতে পারে।
Random Sample দেখলে:

- Data Distribution সম্পর্কে ধারণা পাওয়া যায়
- Random Observation যাচাই করা যায়
- Data Entry সমস্যা বোঝা যায়

head() এবং sample() এর পার্থক্য

head():

- শুরু থেকে Data দেখায়
- Sequential View দেয়

sample():

- Random Data দেখায়
- Better Overall Understanding দেয়

Analytical View নেওয়ার সময় যেসব বিষয় খেয়াল করতে হয়

Dataset দেখার সময় সাধারণত নিচের বিষয়গুলো লক্ষ্য করা হয়:

- Column Name Meaningful কি না
- Missing Value আছে কি না
- Same Type Data একই Format এ আছে কি না
- Numerical এবং Categorical Column আলাদা করা
- কোনো Irrelevant Feature আছে কি না

Titanic Dataset উদাহরণ

Titanic Dataset এ সাধারণত নিচের ধরনের Feature থাকে:

- PassengerId
- Name
- Age
- Sex
- Ticket
- Fare
- Cabin
- Survived

এখন Analytical View এর মাধ্যমে আমরা বুঝতে পারি:

- কোন Feature Numerical
- কোন Feature Categorical
- কোথায় Missing Value থাকতে পারে
- কোন Feature Prediction এর জন্য গুরুত্বপূর্ণ হতে পারে

এই ধাপের মূল উদ্দেশ্য

Analytical View এর মাধ্যমে আমরা Dataset সম্পর্কে একটি Overall Understanding তৈরি করি।
এটি পরবর্তী ধাপগুলো যেমন:

- Feature Screening
- Statistical Profiling
- Univariate Analysis
- Multivariate Analysis

এসব করার ভিত্তি তৈরি করে।

সারাংশ

Analytical View of a Dataset হলো Dataset সম্পর্কে প্রাথমিক বিশ্লেষণ।
এই ধাপে আমরা:

- Dataset এর Shape দেখি
- Total Feature এবং Sample Count দেখি
- প্রথম কিছু Row দেখি
- Random Sample দেখি
- Dataset এর Structure বোঝার চেষ্টা করি

Feature Screening

Feature Screening কী?

EDA এর একটি গুরুত্বপূর্ণ ধাপ হলো Feature Screening।
এই ধাপে Dataset এর বিভিন্ন Feature বা Column পরীক্ষা করা হয় যাতে Data এর ভিতরের সমস্যা শনাক্ত করা যায়।
সহজভাবে বললে, Feature Screening হলো Dataset এর Quality Check করার প্রক্রিয়া।
এখানে সাধারণত নিচের বিষয়গুলো পরীক্ষা করা হয়:

- Missing Value আছে কি না
- Duplicate Data আছে কি না
- Irrelevant Feature আছে কি না
- Data Consistency ঠিক আছে কি না

কেন Feature Screening প্রয়োজন?

যদি Dataset এর ভিতরে সমস্যা থাকে, তাহলে Machine Learning Model ভুল Prediction দিতে পারে।
উদাহরণ:

- Missing Value থাকলে Model Confuse হতে পারে
- Duplicate Data থাকলে Bias তৈরি হতে পারে
- Irrelevant Feature থাকলে Model Performance কমে যেতে পারে

তাই Model তৈরির আগে Feature Screening করা অত্যন্ত গুরুত্বপূর্ণ।

Missing Value Check করা

প্রথমে Dataset এ Missing Value আছে কি না তা পরীক্ষা করা হয়।

```
1df.isnull().sum()
```

isnull() কী করে?

isnull() প্রতিটি Cell পরীক্ষা করে দেখে সেখানে Missing Value আছে কি না।
যদি Missing থাকে:

- True রিটার্ন করে

না থাকলে:

- False রিটার্ন করে

sum() কেন ব্যবহার করা হয়?

কারণ True কে Python এ 1 হিসেবে ধরা হয়।
তাই sum() ব্যবহার করলে প্রতিটি Column এ মোট কতগুলো Missing Value আছে তা পাওয়া যায়।

সম্ভাব্য Output

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
Cabin          687
Embarked         2
dtype: int64
```

এখানে দেখা যাচ্ছে:

- Age Column এ 177 টি Missing Value আছে
- Cabin Column এ 687 টি Missing Value আছে
- Embarked Column এ 2 টি Missing Value আছে

Missing Value কেন সমস্যা?

Missing Value থাকার কারণে:

- Statistical Calculation ভুল হতে পারে

Visualization বিভ্রান্তিকর হতে পারে
Machine Learning Algorithm Error দিতে পারে
Prediction Accuracy কমে যেতে পারে

Missing Value Handle করার কিছু উপায়

সাধারণত Missing Value Handle করার জন্য:

Row Delete করা হয়
Column Delete করা হয়
Mean/Median/Mode দিয়ে Fill করা হয়
Forward Fill বা Backward Fill ব্যবহার করা হয়

তবে কোন Method ব্যবহার করা হবে তা Dataset এর উপর নির্ভর করে।

Duplicate Data Check করা

এখন Dataset এ Duplicate Row আছে কি না তা পরীক্ষা করা হবে।

```
1df.duplicated().sum()
```

duplicated() কী করে?

duplicated() প্রতিটি Row পরীক্ষা করে দেখে সেটি আগে থেকে Dataset এ আছে কি না।
যদি Duplicate হয়:

True দেয়

না হলে:

False দেয়

sum() কীভাবে কাজ করছে?

এখানেও True কে 1 হিসেবে ধরা হয়।
তাই sum() ব্যবহার করলে মোট Duplicate Row সংখ্যা পাওয়া যায়।

সম্ভাব্য Output

```
1107
```

অর্থাৎ Dataset এ 107 টি Duplicate Row আছে।

Duplicate Data কেন সমস্যা?

Duplicate Data থাকার কারণে:

- Model একই তথ্য বারবার শিখে ফেলে
- Bias তৈরি হয়
- Statistical Result ভুল হতে পারে
- Dataset এর বাস্তব Distribution নষ্ট হতে পারে

Duplicate Row Remove করা

Duplicate Data মুছে ফেলার জন্য drop_duplicates() ব্যবহার করা হয়।

```
df.drop_duplicates(inplace=True)
```

drop_duplicates() কী করে?

এই Method Dataset থেকে Duplicate Row Remove করে দেয়।

inplace=True কেন ব্যবহার করা হয়?

inplace=True ব্যবহার করলে Original Dataset এর মধ্যেই পরিবর্তন হয়ে যায়।
অর্থাৎ:

- নতুন Dataset তৈরি হয় না
- df সরাসরি Update হয়ে যায়

Duplicate Remove করার পর আবার Check করা

Duplicate Remove হয়েছে কি না তা আবার পরীক্ষা করা যেতে পারে।

```
df.duplicated().sum()
```

সম্ভাব্য Output:

```
0
```

অর্থাৎ এখন আর কোনো Duplicate Row নেই।

Feature Screening এ আর কী কী দেখা হয়?

বাস্তব প্রজেক্টে Feature Screening এর সময় আরও কিছু বিষয় দেখা হয়:

- Incorrect Data Type
- Constant Feature
- High Cardinality
- Irrelevant Feature
- Unique Identifier
- Data Imbalance

Titanic Dataset এর ক্ষেত্রে পর্যবেক্ষণ

Titanic Dataset এ সাধারণত:

- Cabin এ অনেক Missing Value থাকে
- Age এ কিছু Missing Value থাকে
- Duplicate Row থাকতে পারে
- PassengerId অনেক সময় Prediction এর জন্য গুরুত্বপূর্ণ নয়

এসব বিষয় Feature Screening এর মাধ্যমে শনাক্ত করা যায়।

Feature Screening এর মূল উদ্দেশ্য

এই ধাপের প্রধান লক্ষ্য হলো:

- Dataset Clean করা
- Problematic Feature শনাক্ত করা
- Better Analysis এর জন্য Data প্রস্তুত করা

সারাংশ

Feature Screening হলো Dataset এর Feature বা Column গুলো পরীক্ষা করার প্রক্রিয়া।
এই ধাপে আমরা:

- Missing Value Check করি
- Duplicate Data শনাক্ত করি
- Duplicate Remove করি
- Dataset এর Quality যাচাই করি

Statistical Profiling

Statistical Profiling কী?

EDA এর একটি গুরুত্বপূর্ণ ধাপ হলো Statistical Profiling।
এই ধাপে Dataset এর বিভিন্ন Statistical Information বিশ্লেষণ করা হয় যাতে Data সম্পর্কে গভীর ধারণা পাওয়া যায়।
সহজভাবে বললে, Statistical Profiling হলো Dataset এর সংখ্যাগত এবং গঠনগত বৈশিষ্ট্য বিশ্লেষণ করার প্রক্রিয়া।
এই ধাপে সাধারণত নিচের বিষয়গুলো দেখা হয়:

- Data Type
- Non-null Value Count
- Mean
- Median
- Minimum Value
- Maximum Value
- Standard Deviation
- Data Distribution সম্পর্কে ধারণা

কেন Statistical Profiling গুরুত্বপূর্ণ?

Statistical Profiling এর মাধ্যমে আমরা বুঝতে পারি:

- কোন Column Numerical
- কোন Column Categorical
- Data এর Range কেমন
- Data Balanced নাকি Skewed
- Outlier থাকার সম্ভাবনা আছে কি না

এটি পরবর্তী Analysis এবং Machine Learning এর জন্য অত্যন্ত গুরুত্বপূর্ণ।

Dataset Information দেখা

Dataset সম্পর্কে Basic Structural Information দেখার জন্য info() ব্যবহার করা হয়।

```
1df.info()
```

info() কী দেখায়?

info() সাধারণত নিচের তথ্যগুলো দেখায়:

- Total Row Count
- Total Column Count
- Column Name
- Data Type
- Non-null Value Count
- Memory Usage

Output সম্পর্কে ধারণা

ধরুন Titanic Dataset এর জন্য Output এ নিচের ধরনের তথ্য দেখা গেল:

- Age → float64
- Fare → float64
- Survived → int64
- Sex → object
- Cabin → object

এখান থেকে বোঝা যায়:

- float64 → Decimal Numerical Data
- int64 → Integer Numerical Data
- object → Text বা Categorical Data

Data Type কেন গুরুত্বপূর্ণ?

কারণ প্রতিটি Data Type এর জন্য Analysis Method আলাদা হয়।
উদাহরণ:
Numerical Column এর জন্য:

- Mean
- Median
- Histogram

Categorical Column এর জন্য:

- Countplot
- Value Counts

ব্যবহার করা হয়।

Non-null Count কেন গুরুত্বপূর্ণ?

Non-null Count দেখে বোঝা যায় কোন Column এ Missing Value আছে।
উদাহরণ:
যদি কোনো Column এ:

- Total Row = 891
- Non-null = 714

তাহলে বোঝা যায় সেখানে Missing Value রয়েছে।

Memory Usage

info() Dataset এর Memory Usage ও দেখায়।
এটি বড় Dataset এর ক্ষেত্রে গুরুত্বপূর্ণ কারণ:

- Large Dataset বেশি RAM ব্যবহার করে
- Optimization প্রয়োজন হতে পারে

Statistical Summary দেখা

Numerical Column এর Statistical Summary দেখার জন্য describe() ব্যবহার করা হয়।

```
1df.describe()
```

describe() কী দেখায়?

describe() সাধারণত Numerical Column এর জন্য নিচের Statistical Information দেখায়:

- count
- mean
- std
- min
- 25%

50%
75%
max

count

count দেখায় কতগুলো Non-null Value আছে।
উদাহরণ:
যদি Age Column এর count = 714 হয়,
তাহলে বোঝা যায় বাকি Row গুলোতে Missing Value আছে।

mean

mean হলো Average Value।
সূত্র:
$$\text{Mean} = \frac{\sum x}{n}$$
উদাহরণ:
যদি Passenger Age এর Mean = 29.7 হয়,
তাহলে Average Passenger Age প্রায় 30 বছর।

std (Standard Deviation)

Standard Deviation দেখায় Data কতটা Spread হয়েছে।
কম Standard Deviation:

Data কাছাকাছি আছে

বেশি Standard Deviation:

Data বেশি ছড়িয়ে আছে

min এবং max

min = সবচেয়ে ছোট Value
max = সবচেয়ে বড় Value

এগুলোর মাধ্যমে Data Range বোঝা যায়।

Percentile কী?

describe() এ 25%, 50%, এবং 75% দেখানো হয়।
এগুলোকে Percentile বলা হয়।

25% (First Quartile)

২৫% Data এই Value এর নিচে থাকে।

50% (Median)

৫০% Data এই Value এর নিচে থাকে।
এটি Median হিসেবেও পরিচিত।

75% (Third Quartile)

৭৫% Data এই Value এর নিচে থাকে।

কেন Percentile গুরুত্বপূর্ণ?

Percentile ব্যবহার করে:

- Data Distribution বোঝা যায়
- Outlier শনাক্ত করা যায়
- Skewness সম্পর্কে ধারণা পাওয়া যায়

Outlier সম্পর্কে ধারণা

যদি max Value অনেক বেশি হয় এবং 75% এর কাছাকাছি না থাকে,
তাহলে Outlier থাকার সম্ভাবনা থাকে।
উদাহরণ:

- 75% Fare = 31
- max Fare = 512

এখানে Fare Column এ Outlier থাকতে পারে।

Titanic Dataset এ Statistical Profiling

Titanic Dataset এ Statistical Profiling করলে সাধারণত দেখা যায়:

- Age এ Missing Value আছে
- Fare Column এ Outlier থাকতে পারে
- Passenger Class Integer Type
- Sex এবং Embarked Categorical Type

info() এবং describe() এর পার্থক্য

info():

- Dataset Structure দেখায়
- Data Type দেখায়
- Missing Value সম্পর্কে ধারণা দেয়

describe():

- Statistical Summary দেখায়
- Numerical Analysis দেয়

Statistical Profiling এর মূল উদ্দেশ্য

এই ধাপের প্রধান লক্ষ্য হলো:

- Dataset এর গঠন বোঝা
- Numerical Feature বিশ্লেষণ করা
- Missing Value শনাক্ত করা
- Outlier সম্পর্কে ধারণা পাওয়া
- Data Distribution বোঝা

সারাংশ

Statistical Profiling হলো Dataset এর Statistical এবং Structural বিশ্লেষণ।
এই ধাপে আমরা:

- info() ব্যবহার করে Dataset Structure দেখি
- describe() ব্যবহার করে Statistical Summary দেখি

Data Type বুঝি
Missing Value শনাক্ত করি
Data Distribution সম্পর্কে ধারণা পাই

Univariate Analysis (Categorical Value Analysis)

Univariate Analysis কী?

Univariate Analysis হলো এমন একটি Analysis যেখানে একবারে শুধুমাত্র একটি Feature বা Column বিশ্লেষণ করা হয়।
এখানে আমরা একটি Column এর Distribution, Frequency এবং Pattern বোঝার চেষ্টা করি।
যদি Column টি Categorical হয়, তাহলে সাধারণত দেখা হয়:

- কোন Category কতবার এসেছে
- কোন Category বেশি বা কম
- Percentage Distribution কেমন
- Data Balanced নাকি Imbalanced

Categorical Data কী?

Categorical Data হলো এমন Data যা বিভিন্ন Category বা Group নির্দেশ করে।
উদাহরণ:

- Male / Female
- Yes / No
- Survived / Not Survived
- 1st Class / 2nd Class / 3rd Class

Titanic Dataset এ কিছু Categorical Feature হলো:

- Survived
- Pclass
- Sex
- Embarked

কেন Categorical Analysis গুরুত্বপূর্ণ?

Categorical Analysis এর মাধ্যমে আমরা বুঝতে পারি:

- কোন Category Dominant
- Data Imbalance আছে কি না
- কোন Category Rare
- Feature সম্পর্কে প্রাথমিক Insight

এটি ভবিষ্যতে Model Building এবং Feature Engineering এ সাহায্য করে।

Survived Column Analysis

প্রথমে Survived Column বিশ্লেষণ করা যাক।

```
1sns.countplot(data = df , x=df['Survived'])
```

countplot() কী করে?

countplot() প্রতিটি Category কতবার এসেছে তা Graph আকারে দেখায়।
এখানে:

- x-axis এ Category থাকে
- y-axis এ Count থাকে

Survived Column এর অর্থ

Titanic Dataset এ সাধারণত:

- 0 → Passenger Survive করেনি
- 1 → Passenger Survive করেছে

Countplot থেকে কী বোঝা যায়?

Countplot দেখে বোঝা যায়:

- কোন Category বেশি
- Data Balanced নাকি না
- Distribution কেমন

যদি 0 এর Count অনেক বেশি হয়,
তাহলে বোঝা যায় বেশি Passenger বাঁচেনি।

Percentage Calculation

শুধু Count দেখলে সবসময় পরিষ্কার ধারণা পাওয়া যায় না।
তাই Percentage বের করা গুরুত্বপূর্ণ।

```
(df['Survived'].value_counts() / len(df) ) * 100
```

value_counts() কী করে?

value_counts() প্রতিটি Category এর Frequency Count বের করে।
উদাহরণ:

```
df['Survived'].value_counts()
```

সম্ভাব্য Output:

```
0    549
1    342
```

Percentage Formula

Percentage বের করার ধারণা:
\text{Percentage} = \frac{\text{Category Count}}{\text{Total Count}} \times 100
এখানে:

Category Count = নির্দিষ্ট Category এর সংখ্যা
Total Count = মোট Row সংখ্যা

Pie Chart তৈরি করা

এখন Pie Chart ব্যবহার করে Survival Distribution দেখা যাক।

```
survived_count =  df['Survived'].value_counts()

survived_label = df['Survived'].unique()

plt.pie(survived_count, labels = survived_label, autopct="%1.1f")
```

```
6plt.title('Pie chart of survival rate')
```

```
7plt.show()
```

Pie Chart কেন ব্যবহার করা হয়?

Pie Chart ব্যবহার করা হয়:

- Percentage Distribution বোঝার জন্য
- Category Ratio Visualize করার জন্য

এখানে প্রতিটি Slice একটি Category নির্দেশ করে।

autopct="%1.1f" এর কাজ

এটি Pie Chart এ Percentage Show করে।
উদাহরণ:

- 61.6%
- 38.4%

Pclass Column Analysis

এখন Passenger Class বিশ্লেষণ করা যাক।

```
1sns.countplot(data = df , x=df['Pclass'])
```

Pclass কী নির্দেশ করে?

সাধারণত:

- 1 → First Class
- 2 → Second Class
- 3 → Third Class

Percentage বের করা

```
1(df['Pclass'].value_counts() / len(df) ) * 100
```

এখানে বোঝা যায় কোন Class এ কত শতাংশ Passenger ছিল।

Sex Column Analysis

এখন Gender Distribution বিশ্লেষণ করা যাক।

```
1sns.countplot(data = df , x=df['Sex'])
```

Percentage বের করা

```
1(df['Sex'].value_counts() / len(df) ) * 100
```

Sex Analysis থেকে কী বোঝা যায়?

এখান থেকে বোঝা যায়:

Male Passenger বেশি নাকি Female
Gender Distribution Balanced কি না

Embarked Column Analysis

এখন Embarked Feature বিশ্লেষণ করা যাক।

```
1sns.countplot(data = df , x=df['Embarked'])
```

Embarked কী নির্দেশ করে?

Embarked সাধারণত Passenger কোন Port থেকে Titanic এ উঠেছে তা নির্দেশ করে।
উদাহরণ:

- C → Cherbourg
- Q → Queenstown
- S → Southampton

Percentage বের করা

```
1(df['Embarked'].value_counts() / len(df) ) * 100
```

Embarked Analysis থেকে কী বোঝা যায়?

এখানে বোঝা যায়:

- কোন Port থেকে সবচেয়ে বেশি Passenger এসেছে
- কোন Category Rare
- Data Distribution কেমন

কেন Percentage গুরুত্বপূর্ণ?

কারণ শুধু Count সবসময় পরিষ্কার ধারণা দেয় না।
উদাহরণ:

- Dataset ছোট হলে Count কম হবে
- Dataset বড় হলে Count বেশি হবে

কিন্তু Percentage Relative Understanding দেয়।

Countplot বনাম Pie Chart

Countplot:

- Exact Count বোঝার জন্য ভালো
- Category Comparison সহজ

Pie Chart:

- Percentage বোঝার জন্য ভালো
- Ratio Visualization সহজ

Univariate Categorical Analysis এর মূল উদ্দেশ্য

এই Analysis এর মাধ্যমে আমরা:

- Category Distribution বুঝি
- Frequency Analysis করি
- Percentage Distribution দেখি
- Imbalance শনাক্ত করি
- Initial Insight পাই

সারাংশ

Categorical Univariate Analysis এ একটি Categorical Feature আলাদাভাবে বিশ্লেষণ করা হয়।
এই ধাপে আমরা:

- countplot ব্যবহার করি
- value_counts() ব্যবহার করি
- Percentage বের করি
- Pie Chart তৈরি করি
- Distribution এবং Imbalance বিশ্লেষণ করি

Univariate Analysis (Numerical Column Analysis)

Numerical Univariate Analysis কী?

Numerical Univariate Analysis হলো এমন একটি Analysis যেখানে একটি Numerical Feature আলাদাভাবে বিশ্লেষণ করা হয়।
এখানে আমরা সাধারণত নিচের বিষয়গুলো বোঝার চেষ্টা করি:

- Data Distribution কেমন
- Data Symmetric নাকি Skewed
- Outlier আছে কি না
- Data কোন Range এ বেশি Concentrated
- Spread কেমন

Numerical Data কী?

যেসব Data সংখ্যা দ্বারা প্রকাশ করা যায় সেগুলো Numerical Data।
উদাহরণ:

- Age
- Salary
- Fare
- Height
- Weight

Titanic Dataset এ Numerical Feature এর উদাহরণ:

- Age
- Fare

Numerical Analysis কেন গুরুত্বপূর্ণ?

Numerical Analysis এর মাধ্যমে আমরা বুঝতে পারি:

- Data Distribution কেমন
- Outlier আছে কি না
- Data Normal কিনা
- Model এর জন্য Transformation প্রয়োজন কি না

এটি Feature Engineering এবং Model Selection এ গুরুত্বপূর্ণ ভূমিকা রাখে।

Age Column এর Histogram

প্রথমে Age Column এর Histogram তৈরি করা যাক।

```
1sns.histplot(df['Age'],bins=50)


2

3plt.title('Histogram of Age')


4plt.xlabel('Ages')


5plt.ylabel('frequency')


6

7plt.show()
```

Histogram কী?

Histogram হলো এমন একটি Graph যা Numerical Data এর Frequency Distribution দেখায়।
এখানে:

- x-axis → Numerical Value
- y-axis → Frequency

bins=50 এর অর্থ কী?

Histogram এ Data কে কয়েকটি Group বা Interval এ ভাগ করা হয়।
এই Group গুলোকে Bin বলা হয়।
এখানে:

```
1bins=50
```

অর্থাৎ Data কে 50 টি Interval এ ভাগ করা হয়েছে।

Histogram থেকে কী বোঝা যায়?

Histogram দেখে বোঝা যায়:

- কোন Range এ Data বেশি
- Distribution Symmetric নাকি না
- Data Skewed কি না
- Multiple Peak আছে কি না

Age Histogram বিশ্লেষণ

Age Histogram থেকে সাধারণত দেখা যায়:

- মাঝামাঝি বয়সের Passenger বেশি
- খুব কম বয়সী এবং খুব বেশি বয়সী Passenger কম
- Distribution কিছুটা Right Skewed হতে পারে

KDE Plot তৈরি করা

এখন একই Data এর KDE Plot দেখা যাক।

```
sns.kdeplot(df['Age'])


plt.title('KDE plot of Age')


plt.xlabel('Ages')


plt.ylabel('probabilty distribution')


plt.show()
```

KDE Plot কী?

KDE এর পূর্ণরূপ হলো Kernel Density Estimation।
এটি Histogram এর একটি Smooth Version।
KDE Plot Data এর Probability Distribution Smooth Curve আকারে দেখায়।

KDE Plot কেন ব্যবহার করা হয়?

KDE Plot ব্যবহার করা হয় কারণ:

- Distribution আরও Smooth দেখা যায়
- Pattern সহজে বোঝা যায়
- Peak শনাক্ত করা সহজ হয়

Histogram এবং KDE Plot এর পার্থক্য

Histogram:

Bar ব্যবহার করে
Frequency দেখায়

KDE Plot:

Smooth Curve ব্যবহার করে
Probability Density দেখায়

Fare Column এর Histogram

এখন Fare Feature বিশ্লেষণ করা যাক।

```
1sns.histplot(df['Fare'],bins=50)


2

3plt.title('Histogram of Fare')


4plt.xlabel('Fare')


5plt.ylabel('frequency')


6

7plt.show()
```

Fare Distribution থেকে কী বোঝা যায়?

Fare Histogram এ সাধারণত দেখা যায়:

অধিকাংশ Passenger কম Fare দিয়েছে
কিছু Passenger অনেক বেশি Fare দিয়েছে
Distribution অনেক Right Skewed

Right Skewed Distribution কী?

যখন বেশিরভাগ Data ছোট Value এর দিকে থাকে কিন্তু কিছু বড় Value ডানদিকে Tail তৈরি করে, তখন তাকে Right Skewed Distribution বলে।

Fare এর KDE Plot

```
sns.kdeplot(df['Fare'])


plt.title('KDE plot of Fare')


plt.xlabel('Fare')


plt.ylabel('probabilty distribution')


plt.show()
```

KDE Plot থেকে কী বোঝা যায়?

এখানে বোঝা যায়:

- Data কোথায় বেশি Concentrated
- Distribution কতটা Smooth
- Extreme Value আছে কি না

Boxplot তৈরি করা

এখন Fare Column এর Boxplot দেখা যাক।

```
sns.boxplot(df['Fare'])
```

Boxplot কী?

Boxplot হলো এমন একটি Visualization যা Data এর Spread এবং Outlier দেখায়।
এটি সাধারণত নিচের বিষয়গুলো দেখায়:

- Minimum Value
- Maximum Value
- Median
- Quartile
- Outlier

Boxplot এর অংশগুলো

একটি Boxplot এ সাধারণত থাকে:

- Box
- Median Line
- Whisker
- Outlier Point

Median কী?

Median হলো Dataset এর মাঝের Value।
৫০% Data Median এর নিচে থাকে।

Quartile কী?

Quartile Data কে চার ভাগে ভাগ করে।

- Q1 → 25%
- Q2 → 50% (Median)
- Q3 → 75%

Outlier কী?

যেসব Value অন্যান্য Value থেকে অনেক দূরে থাকে সেগুলোকে Outlier বলে।
উদাহরণ:
যদি অধিকাংশ Fare হয়:

10 থেকে 50

কিন্তু কিছু Fare হয়:

300 বা 500

তাহলে সেগুলো Outlier হতে পারে।

Fare Boxplot বিশ্লেষণ

Fare Boxplot এ সাধারণত দেখা যায়:

অনেক Outlier আছে
Distribution Right Skewed
বেশিরভাগ Fare কম Range এ আছে

কেন Outlier গুরুত্বপূর্ণ?

Outlier থাকার কারণে:

Mean পরিবর্তিত হতে পারে
Model Performance খারাপ হতে পারে
Distribution Distorted হতে পারে

তাই Outlier শনাক্ত করা গুরুত্বপূর্ণ।

Numerical Analysis এ ব্যবহৃত Visualization

এই Module এ আমরা তিন ধরনের Visualization ব্যবহার করেছি:

Histogram
KDE Plot
Boxplot

Histogram এর ব্যবহার

Histogram ব্যবহার করা হয়:

Frequency Distribution দেখার জন্য
Data Spread বোঝার জন্য

KDE Plot এর ব্যবহার

KDE Plot ব্যবহার করা হয়:

- Smooth Distribution দেখার জন্য
- Density বোঝার জন্য

Boxplot এর ব্যবহার

Boxplot ব্যবহার করা হয়:

- Outlier শনাক্ত করার জন্য
- Spread এবং Quartile বোঝার জন্য

Numerical Univariate Analysis এর মূল উদ্দেশ্য

এই Analysis এর মাধ্যমে আমরা:

- Distribution বুঝি
- Skewness শনাক্ত করি
- Outlier খুঁজে বের করি
- Data Spread বিশ্লেষণ করি
- Statistical Pattern বুঝি

সারাংশ

Numerical Univariate Analysis এ একটি Numerical Feature আলাদাভাবে বিশ্লেষণ করা হয়।
এই ধাপে আমরা:

- Histogram ব্যবহার করি
- KDE Plot ব্যবহার করি
- Boxplot ব্যবহার করি
- Distribution বিশ্লেষণ করি
- Outlier শনাক্ত করি
- Data Spread বুঝি

এটি Dataset এর Numerical Feature সম্পর্কে গভীর ধারণা প্রদান করে।

Introduction to Multivariate Analysis

আমরা এর আগে Univariate এবং Bivariate Analysis শিখেছি। Univariate Analysis-এ আমরা একটিমাত্র variable নিয়ে কাজ করতাম — যেমন শুধু Age-এর distribution দেখা। Bivariate Analysis-এ দুটো variable-এর মধ্যে সম্পর্ক দেখতাম — যেমন Age এবং Fare। কিন্তু real-world data-তে একটি outcome সাধারণত একটি বা দুটি variable-এর উপর নির্ভর করে না। একসাথে অনেক variable মিলে একটি pattern তৈরি করে। এই জায়গাতেই Multivariate Analysis আসে।

What is Multivariate Analysis?

Multivariate Analysis হলো একসাথে তিন বা তার বেশি variable-এর মধ্যে সম্পর্ক বোঝার পদ্ধতি। এটি আমাদের বলে — শুধু "Age বেশি হলে কী হয়" নয়, বরং "Age বেশি হলে এবং Sex female হলে Survival rate কেমন হয়?" এই ধরনের প্রশ্নের উত্তর Multivariate Analysis দেয়।

উদাহরণ হিসেবে Titanic dataset-এর কথা ভাবো। একজন passenger survive করবে কিনা সেটা নির্ভর করে তার Sex, Pclass, Age, Fare — একসাথে অনেক কিছুর উপর। শুধু একটা variable দেখে সিদ্ধান্ত নিলে আমরা অনেক গুরুত্বপূর্ণ pattern miss করবো।

Univariate vs Bivariate vs Multivariate

Type	Variables	Example	Plot
Univariate	১টি	Age-এর distribution	Histogram, KDE
Bivariate	২টি	Age vs Fare	Scatterplot, Barplot
Multivariate	৩+ টি	Age, Sex, Survived একসাথে	Countplot with Hue, KDE with Hue

Why Do We Need It?

শুধু Bivariate Analysis দিয়ে আমরা দেখতে পারি Pclass এবং Survived-এর মধ্যে সম্পর্ক আছে। কিন্তু Multivariate Analysis দিয়ে আমরা দেখতে পারি — first class-এর female passengers-এর survival rate কত, এবং third class-এর male passengers-এর survival rate কত। এই deeper insight machine learning model বানাতে অনেক কাজে আসে।

সহজ করে বলতে গেলে — Univariate Analysis বলে "কী আছে", Bivariate Analysis বলে "দুটোর মধ্যে সম্পর্ক কী", আর Multivariate Analysis বলে "আসল ছবিটা কেমন।"

What We Will Learn

এই module-এ আমরা Multivariate Analysis-এর জন্য সবচেয়ে বেশি ব্যবহৃত plots শিখবো।

- **Countplot with Hue** — categorical variable-এর frequency তৃতীয় variable দিয়ে ভাগ করে দেখা
- **Barplot** — categorical এবং numerical variable-এর summary তৃতীয় variable সহ দেখা
- **KDE Plot** — continuous variable-এর distribution আলাদা groups-এ compare করা

Scatterplot — দুটো numerical variable-এর relationship hue এবং size দিয়ে আরো গভীরভাবে দেখা
Y Data Profiling — একটি command-এ পুরো dataset-এর automatic analysis

Summary

Multivariate Analysis আমাদের data-র পূর্ণ চিত্র দেখতে সাহায্য করে। একটি বা দুটি variable দিয়ে যে pattern ধরা যায় না, তিনটি বা তার বেশি variable একসাথে দেখলে সেই pattern স্পষ্ট হয়ে ওঠে। এই module জুড়ে আমরা real Titanic data ব্যবহার করে প্রতিটি plot হাতে-কলমে শিখবো।

Countplot with Hue

Countplot আমরা আগে শিখেছি — এটি একটি categorical variable-এর প্রতিটি category কতবার আছে তা bar দিয়ে দেখায়। কিন্তু শুধু count দেখলে অনেক সময় আসল গল্পটা বোঝা যায় না। যেমন Titanic-এ মোট কতজন male আর female ছিল সেটা জানা যায়, কিন্তু তাদের মধ্যে কতজন survive করেছে সেটা জানা যায় না। এই সমস্যার সমাধান হলো `hue` parameter — যা একটি তৃতীয় categorical variable color দিয়ে plot-এ যোগ করে।

What is Hue?

`hue` parameter একটি categorical column নেয় এবং সেই column-এর প্রতিটি unique value-এর জন্য আলাদা color ব্যবহার করে। এতে একটি plot-এই দুটি categorical variable-এর মধ্যে সম্পর্ক একসাথে দেখা যায়।
উদাহরণ হিসেবে ভাবো — x-axis-এ Sex (male/female), আর hue-তে Survived (yes/no)। এতে প্রতিটি gender-এর জন্য দুটো bar দেখাবে — একটি যারা survive করেছে, একটি যারা করেনি।

Example 1: Sex vs Survived

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('titanic_data_updated.csv')

sns.countplot(x=df['Sex'], hue=df['Survived'])


plt.title('Survival Count by Sex')
```

```
plt.show()
```

এই plot থেকে আমরা দেখতে পাই female passengers-এর মধ্যে survive করার হার অনেক বেশি ছিল male-দের তুলনায়। শুধু countplot দিয়ে এই comparison সম্ভব হতো না।
groupby দিয়ে exact percentage বের করা যায়:

```
gender_split = df.groupby('Sex')['Survived'].value_counts(normalize=True)


print(gender_split)
```

`normalize=True` দিলে count-এর বদলে proportion (0 থেকে 1) দেখায়। এতে বোঝা যায় female-দের মধ্যে কত % survive করেছে এবং male-দের মধ্যে কত % করেছে।

Example 2: Pclass vs Survived

```
sns.countplot(x=df['Pclass'], hue=df['Survived'])

plt.title('Survival Count by Passenger Class')
plt.show()
```

এই plot থেকে দেখা যায় first class passengers সবচেয়ে বেশি survive করেছে এবং third class সবচেয়ে কম। Passenger class এবং survival-এর মধ্যে একটি স্পষ্ট pattern আছে।

```
pclass_split =
df.groupby('Pclass')['Survived'].value_counts(normalize=True)
print(pclass_split)
```

When to Use Countplot with Hue?

Countplot with Hue তখনই use করবো যখন:

- দুটো categorical variable-এর মধ্যে সম্পর্ক দেখতে চাই
- একটি category-র মধ্যে আরেকটি category-র distribution জানতে চাই
- Groups-এর মধ্যে frequency compare করতে চাই

Summary

`hue` parameter Countplot-কে Bivariate থেকে Multivariate-এ নিয়ে যায়। Titanic data-তে আমরা দেখলাম Sex এবং Pclass উভয়ই Survival-এর সাথে সম্পর্কিত — এবং এই সম্পর্ক Countplot with Hue ছাড়া এত সহজে বোঝা সম্ভব হতো না। `groupby` দিয়ে exact proportion বের করে plot-এর insight verify করা যায়।

Barplot for Cat-Num Analysis

আমরা আগের page-এ দেখলাম Countplot with Hue দিয়ে দুটো categorical variable-এর মধ্যে সম্পর্ক বোঝা যায়। কিন্তু যখন একটি categorical variable এবং একটি numerical variable-এর মধ্যে সম্পর্ক দেখতে হয়, তখন Barplot ব্যবহার করা হয়। Cat-Num মানে Categorical-Numerical — এই ধরনের analysis-এ Barplot সবচেয়ে কার্যকর tool।

What Does Barplot Show?

Seaborn-এর Barplot প্রতিটি categorical group-এর জন্য একটি numerical variable-এর **mean** (গড়) দেখায়। প্রতিটি bar-এর উপরে একটি ছোট line থাকে যাকে **confidence interval** বলে — এটি বলে দেয় mean-এর আশেপাশে data কতটা spread করা আছে।

Example 1: Pclass vs Fare

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('titanic_data_updated.csv')

sns.barplot(x=df['Pclass'], y=df['Fare'])

plt.title('Average Fare by Passenger Class')
plt.show()
```

এই plot থেকে দেখা যায় first class passengers গড়ে সবচেয়ে বেশি Fare দিয়েছে এবং third class সবচেয়ে কম। x-axis-এ Pclass (categorical) এবং y-axis-এ Fare (numerical) — এটাই Cat-Num analysis-এর basic structure।

Example 2: Survived vs Age with Hue

Barplot-এও `hue` ব্যবহার করা যায়। এতে তিনটি variable একসাথে দেখা যায় — একটি categorical (x-axis), একটি numerical (y-axis), এবং আরেকটি categorical (hue)।

```
sns.barplot(x=df['Survived'], y=df['Age'], hue=df['Sex'])
```

```
2
3plt.title('Average Age of Survivors by Sex')
4plt.show()
```

এই plot থেকে বোঝা যায় — যারা survive করেছে এবং যারা করেনি, তাদের গড় Age কত ছিল এবং এই pattern male ও female-এর মধ্যে কেমন আলাদা।

hue ছাড়া শুধু Survived vs Age দেখলে:

```
1sns.barplot(x=df['Survived'], y=df['Age'])
2
3plt.title('Average Age by Survival')
4plt.show()
```

এই দুটো plot compare করলে বোঝা যায় `hue` যোগ করলে কতটা বেশি information পাওয়া যায়।

Barplot vs Countplot

	Countplot	Barplot
x-axis	Categorical	Categorical
y-axis	Count (automatic)	Numerical (manual)
দেখায়	Frequency	Mean of numerical variable
hue support	✅	✅
কখন use করবো	Category কতবার আছে জানতে	Category অনুযায়ী numerical value-এর গড় জানতে

When to Use Barplot?

Barplot তখন use করবো যখন:

> জানতে চাই বিভিন্ন group-এর numerical variable-এর গড় কত
> যেমন — প্রতিটি Pclass-এর গড় Fare কত?
> যেমন — survive করা এবং না করা passengers-এর গড় Age কত?

Summary

Barplot Cat-Num analysis-এর সবচেয়ে সহজ এবং কার্যকর tool। এটি categorical group অনুযায়ী numerical variable-এর mean দেখায়। `hue` যোগ করলে একসাথে তিনটি variable visualize করা যায় যা data-র আরো গভীর

pattern বুঝতে সাহায্য করে। Titanic data-তে আমরা দেখলাম Pclass, Fare, Survived এবং Age — এই চারটি variable একসাথে Barplot দিয়ে analyze করা সম্ভব হলো।

KDE Plot and Density Graph

আমরা এর আগে Histogram দেখেছি — যেটি একটি numerical variable-এর distribution বোঝার জন্য ব্যবহার হয়। Histogram-এ data-কে ছোট ছোট bins-এ ভাগ করে bar দিয়ে দেখানো হয়। কিন্তু Histogram-এর একটি সমস্যা আছে — bin size পরিবর্তন করলে distribution-এর shape পরিবর্তন হয়ে যায়। এই সমস্যার সমাধান হলো KDE Plot।

What is KDE Plot?

KDE মানে **Kernel Density Estimation**। এটি Histogram-এর মতোই একটি numerical variable-এর distribution দেখায়, কিন্তু bars-এর বদলে একটি smooth continuous curve তৈরি করে। এই curve দেখে বোঝা যায় data কোন range-এ বেশি concentrated এবং কোথায় কম।

সহজ উদাহরণ দিয়ে বলতে গেলে — ধরো একটি ক্লাসের সব ছাত্রের বয়স plot করলে। Histogram দেখাবে প্রতিটি age range-এ কতজন আছে bar দিয়ে। KDE plot দেখাবে সেই একই information কিন্তু একটি smooth wave-এর মতো curve দিয়ে — যা পড়তে অনেক সহজ।

KDE Plot with Hue

KDE plot-এর সবচেয়ে শক্তিশালী ব্যবহার হলো `hue` parameter দিয়ে। এতে আলাদা আলাদা category-র জন্য আলাদা KDE curve দেখা যায় এবং সেগুলো একসাথে compare করা যায়।

Titanic data-তে Age-এর distribution survive করা এবং না করা passengers-এর মধ্যে কেমন আলাদা সেটা দেখি:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('titanic_data_updated.csv')

sns.kdeplot(data=df, x='Age', hue='Survived')

plt.title('Age Distribution by Survival')
plt.show()
```

এই plot থেকে দেখা যায়:

- যারা survive করেছে তাদের মধ্যে শিশু (ছোট বয়স) বেশি ছিল
- যারা survive করেনি তাদের মধ্যে মধ্যবয়সী passengers বেশি ছিল
- দুটো group-এর Age distribution-এর shape আলাদা

এই insight শুধু Histogram দিয়ে এত সহজে বোঝা যেত না।

KDE vs Histogram

	Histogram	KDE Plot
Output	Discrete bars	Smooth continuous curve
Bin dependency	হ্যাঁ — bin size বদলালে shape বদলায়	নেই
Multiple groups compare	কঠিন	সহজ — hue দিয়ে overlay করা যায়
Y-axis	Count বা frequency	Density (0 থেকে 1 এর মধ্যে)
কখন use করবো	Exact count দেখতে	Distribution-এর shape বুঝতে

Understanding the Y-axis

KDE plot-এর y-axis **density** দেখায়, count নয়। Density মানে হলো — curve-এর নিচের মোট area সবসময় 1 হয়। যেখানে curve উঁচু, সেখানে data বেশি concentrated। যেখানে curve নিচু, সেখানে data কম।

এটা একটু confusing মনে হতে পারে শুরুতে। সহজ করে মনে রাখো — **curve** যত উঁচু, সেই **range-**এ তত বেশি **data** আছে।

When to Use KDE Plot?

KDE plot তখন use করবো যখন:

- একটি continuous numerical variable-এর distribution-এর shape দেখতে চাই
- দুই বা তার বেশি group-এর distribution compare করতে চাই
- Data কোন range-এ বেশি concentrated সেটা smooth curve দিয়ে বুঝতে চাই

Summary

KDE Plot হলো Histogram-এর একটি উন্নত version যা smooth curve দিয়ে data-র distribution দেখায়। `hue` parameter যোগ করলে এটি Multivariate Analysis-এর একটি শক্তিশালী tool হয়ে ওঠে। Titanic data-তে আমরা দেখলাম survive করা এবং না করা passengers-এর Age distribution কতটা আলাদা — যা Barplot বা Countplot দিয়ে এতটা clearly বোঝানো সম্ভব হতো না।

Scatterplot

এখন পর্যন্ত আমরা যে plots শিখেছি — Countplot, Barplot, KDE Plot — সেগুলো মূলত categorical variable-কে কেন্দ্র করে। কিন্তু যখন দুটো numerical variable-এর মধ্যে সম্পর্ক দেখতে হয়, তখন Scatterplot সবচেয়ে কার্যকর tool। Scatterplot-এ প্রতিটি data point একটি dot হিসেবে দেখা যায় এবং dots-এর pattern দেখে দুটো variable-এর মধ্যে সম্পর্ক বোঝা যায়।

What is a Scatterplot?

Scatterplot-এ x-axis-এ একটি numerical variable এবং y-axis-এ আরেকটি numerical variable রাখা হয়। Dataset-এর প্রতিটি row একটি dot হিসেবে plot-এ দেখা যায়। dots-এর overall pattern দেখে বোঝা যায় দুটো variable একসাথে বাড়ে কিনা, একটি বাড়লে অন্যটি কমে কিনা, বা কোনো সম্পর্কই নেই।
এই page-এ আমরা Titanic-এর বদলে Seaborn-এর built-in **tips dataset** ব্যবহার করবো। এই dataset-এ একটি restaurant-এর customer-দের bill এবং tip-এর data আছে।

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('tips')
df
```

Dataset-এ আছে: `total_bill`, `tip`, `sex`, `smoker`, `day`, `time`, `size` — এই columns।

Basic Scatterplot

```
sns.scatterplot(data=df,
                x='total_bill',
                y='tip')

plt.title('Total Bill vs Tip')
plt.show()
```

এই plot থেকে দেখা যায় total bill বাড়লে সাধারণত tip-ও বাড়ে — দুটো variable-এর মধ্যে একটি positive সম্পর্ক আছে।

Scatterplot with Hue and Size

Scatterplot-এ `hue` এবং `size` — দুটো extra parameter যোগ করা যায়। এতে একটি plot-এই চারটি variable একসাথে দেখানো সম্ভব হয়।

```
sns.scatterplot(data=df,
                x='total_bill',
                y='tip',
                hue='sex',
                size='size')

plt.title('Total Bill vs Tip by Sex and Party Size')
plt.show()
```

এখানে:

- **x-axis** → total_bill (numerical)
- **y-axis** → tip (numerical)
- **hue** → sex (categorical) — male ও female আলাদা color-এ দেখাবে
- **size** → size (numerical) — party-তে কতজন ছিল, সেই অনুযায়ী dot-এর size বড়-ছোট হবে

এই একটি plot থেকে আমরা একসাথে জানতে পারি — bill এবং tip-এর সম্পর্ক, male ও female-এর pattern আলাদা কিনা, এবং বড় party-তে tip বেশি দেওয়া হয় কিনা।

Reading a Scatterplot

Pattern	মানে
Dots উপরের দিকে যাচ্ছে (↗)	Positive relationship — একটি বাড়লে অন্যটিও বাড়ে
Dots নিচের দিকে যাচ্ছে (↘)	Negative relationship — একটি বাড়লে অন্যটি কমে
Dots randomly ছড়ানো	কোনো সম্পর্ক নেই
Dots একটি curve তৈরি করছে	Non-linear relationship আছে

When to Use Scatterplot?
Scatterplot তখন use করবো যখন:

- দুটো numerical variable-এর মধ্যে সম্পর্ক আছে কিনা দেখতে চাই
- সেই সম্পর্ক কোনো categorical variable-এর জন্য আলাদা কিনা দেখতে চাই
- Data-তে outlier আছে কিনা visually check করতে চাই

Summary

Scatterplot দুটো numerical variable-এর মধ্যে সম্পর্ক dot দিয়ে দেখায়। `hue` এবং `size` parameter যোগ করলে এটি একটি শক্তিশালী Multivariate Analysis tool হয়ে ওঠে। Tips dataset-এ আমরা দেখলাম total bill এবং tip-এর মধ্যে positive সম্পর্ক আছে এবং `hue=sex` ও `size=size` যোগ করে আরো গভীর insight পাওয়া গেল।

Y Data Profiling

এখন পর্যন্ত আমরা আলাদা আলাদা plot বানিয়ে data analyze করেছি। প্রতিটি plot-এর জন্য আলাদা code লিখতে হয়েছে। কিন্তু একটি নতুন dataset পেলে প্রথমেই কী করা উচিত? কতগুলো column আছে, কোথায় missing values আছে, কোন column-এর distribution কেমন — এই সব একসাথে দেখতে অনেক সময় লাগে। Y Data Profiling এই কাজটি মাত্র কয়েক line code-এ করে দেয়।

What is Y Data Profiling?

Y Data Profiling (আগের নাম ছিল Pandas Profiling) একটি Python library যা একটি DataFrame নিয়ে automatically একটি complete EDA report তৈরি করে। এই report-এ থাকে:

- প্রতিটি column-এর data type এবং basic statistics (mean, median, min, max)
- Missing values কোথায় কতটুকু আছে
- প্রতিটি numerical column-এর distribution
- Columns-এর মধ্যে correlation
- Duplicate rows আছে কিনা
- Outliers-এর সংখ্যা

Installation

Y Data Profiling by default installed থাকে না। প্রথমে install করতে হবে:

```
pip install ydata_profiling
```

Colab-এ use করলে cell-এর শুরুতে `!` দিতে হবে:

```
!pip install ydata_profiling
```

Generating the Report

Install হয়ে গেলে মাত্র কয়েক line code-এ report তৈরি করা যায়:

```
import pandas as pd
from ydata_profiling import ProfileReport

df = pd.read_csv('titanic_data_updated.csv')

profile = ProfileReport(df, title='Titanic Profile Report')
```

```
7

8profile.to_file('my_report.html')
```

এখানে:

- `ProfileReport(df, title=...)` → DataFrame নিয়ে report object তৈরি করে
- `title=...` → report-এর উপরে যে নাম দেখাবে
- `profile.to_file('my_report.html')` → report-টি HTML file হিসেবে save করে

Reading the Report

Report generate হলে `my_report.html` নামে একটি file তৈরি হবে। এই file browser-এ open করলে একটি interactive report দেখা যাবে। Report-এ কয়েকটি section থাকে:

Section	কী দেখায়
Overview	Dataset-এর মোট rows, columns, missing values-এর summary
Variables	প্রতিটি column-এর আলাদা analysis — distribution, statistics
Interactions	দুটো numerical column-এর মধ্যে scatterplot
Correlations	সব columns-এর মধ্যে correlation heatmap
Missing Values	কোন column-এ কতটা data missing
Sample	Dataset-এর প্রথম এবং শেষ কয়েকটি row

Y Data Profiling vs Manual EDA

	Manual EDA	Y Data Profiling
সময়	অনেক বেশি — প্রতিটি plot আলাদা করে বানাতে হয়	কয়েক সেকেন্ড
Code	অনেক line	মাত্র ৩-৪ line
Coverage	যা plot করবো শুধু তাই দেখাবে	সব columns automatically analyze হয়
কখন use করবো	নির্দিষ্ট analysis দরকার হলে	নতুন dataset পেলে প্রথমেই overview নিতে

When to Use Y Data Profiling?

Y Data Profiling সবচেয়ে বেশি কাজে আসে যখন:

- নতুন একটি dataset পেয়েছি এবং প্রথমে একটি quick overview দরকার

Dataset-এ কোথায় missing values আছে দ্রুত জানতে চাই
সব columns-এর distribution একসাথে দেখতে চাই
Client বা teammate-কে data-র summary দেখাতে চাই

Summary

Y Data Profiling manual EDA-এর সময় এবং effort অনেক কমিয়ে দেয়। একটি নতুন dataset পেলে সবার আগে `ProfileReport` চালানো একটি ভালো অভ্যাস — এতে data সম্পর্কে প্রাথমিক ধারণা দ্রুত পাওয়া যায় এবং পরবর্তী analysis-এর direction ঠিক করা সহজ হয়।

Module Summary

এই module-এ আমরা Multivariate Analysis-এর সবচেয়ে গুরুত্বপূর্ণ plots এবং tools শিখেছি। প্রতিটি tool আলাদা পরিস্থিতিতে কাজে আসে। এই page-এ সব কিছু একসাথে recap করবো এবং একটি quick reference তৈরি করবো যাতে কোন situation-এ কোন plot use করতে হবে সেটা সহজে মনে রাখা যায়।

What We Learned

Countplot with Hue দিয়ে আমরা দেখলাম দুটো categorical variable-এর মধ্যে সম্পর্ক। Titanic data-তে Sex এবং Pclass — উভয়ই Survival-এর সাথে কতটা সম্পর্কিত সেটা color দিয়ে আলাদা করে বোঝা গেল।

Barplot দিয়ে আমরা দেখলাম categorical group অনুযায়ী numerical variable-এর mean। Pclass অনুযায়ী গড় Fare কত এবং Survived অনুযায়ী গড় Age কত — এই ধরনের প্রশ্নের উত্তর Barplot সহজে দেয়।

KDE Plot দিয়ে আমরা দেখলাম continuous variable-এর smooth distribution। `hue` যোগ করে survive করা এবং না করা passengers-এর Age distribution compare করলাম এবং দুটো group-এর মধ্যে স্পষ্ট পার্থক্য দেখতে পেলাম।

Scatterplot দিয়ে আমরা দেখলাম দুটো numerical variable-এর মধ্যে সম্পর্ক। Tips dataset-এ total bill এবং tip-এর মধ্যে positive relationship ছিল এবং `hue` ও `size` যোগ করে আরো গভীর insight পেলাম।

Y Data Profiling দিয়ে আমরা দেখলাম কীভাবে মাত্র কয়েক line code-এ পুরো dataset-এর automatic EDA report তৈরি করা যায়।

Quick Reference: Which Plot When?

পরিস্থিতি	কোন Plot	Example
দুটো categorical variable compare করতে চাই	Countplot with Hue	Sex vs Survived
Categorical group-এর numerical mean দেখতে চাই	Barplot	Pclass vs Fare
Categorical group-এর numerical mean তৃতীয় variable সহ দেখতে চাই	Barplot with Hue	Survived vs Age, hue=Sex
Continuous variable-এর distribution দেখতে চাই	KDE Plot	Age distribution
দুটো group-এর distribution compare করতে চাই	KDE Plot with Hue	Age by Survived
দুটো numerical variable-এর সম্পর্ক দেখতে চাই	Scatterplot	total_bill vs tip
নতুন dataset-এর quick overview নিতে চাই	Y Data Profiling	Titanic full report

Variable Type Guide

কোন ধরনের data-তে কোন plot কাজ করে:

পরিস্থিতি	কোন Plot	Example
দুটো categorical variable compare করতে চাই	Countplot with Hue	Sex vs Survived
Categorical group-এর numerical mean দেখতে চাই	Barplot	Pclass vs Fare
Categorical group-এর numerical mean তৃতীয় variable সহ দেখতে চাই	Barplot with Hue	Survived vs Age, hue=Sex
Continuous variable-এর distribution দেখতে চাই	KDE Plot	Age distribution
দুটো group-এর distribution compare করতে চাই	KDE Plot with Hue	Age by Survived
দুটো numerical variable-এর সম্পর্ক দেখতে চাই	Scatterplot	total_bill vs tip
নতুন dataset-এর quick overview নিতে চাই	Y Data Profiling	Titanic full report

Key Takeaways

Multivariate Analysis-এর মূল কথা হলো — data-র পূর্ণ চিত্র দেখতে হলে একসাথে তিন বা তার বেশি variable analyze করতে হয়। `hue` parameter প্রায় সব Seaborn plot-এ কাজ করে এবং এটি দিয়ে যেকোনো Bivariate analysis-কে Multivariate-এ নিয়ে যাওয়া যায়।
নতুন dataset পেলে সবার আগে Y Data Profiling দিয়ে overview নাও, তারপর specific questions-এর জন্য আলাদা আলাদা plot বানাও।